

salesforce

Phoenix Thin Client Now! Why and How?

September 2024

Kadir Ozdemir





Thank you



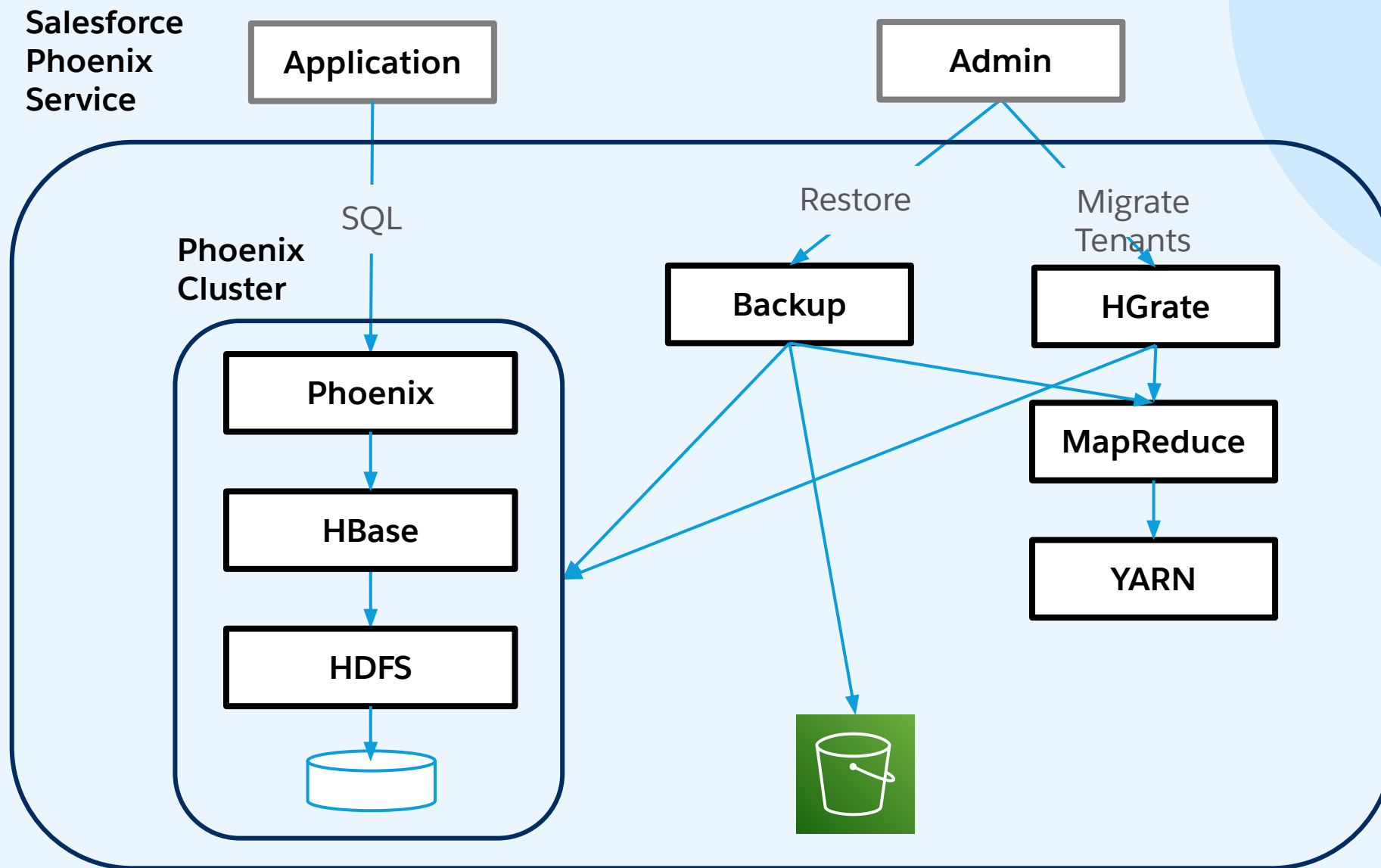
Outline



- Overview
 - HBase and Phoenix Architecture
- Thick Client Problem in Phoenix
- Solution for Thin Client



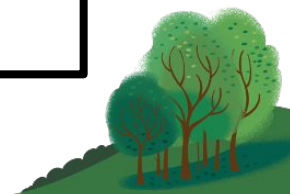
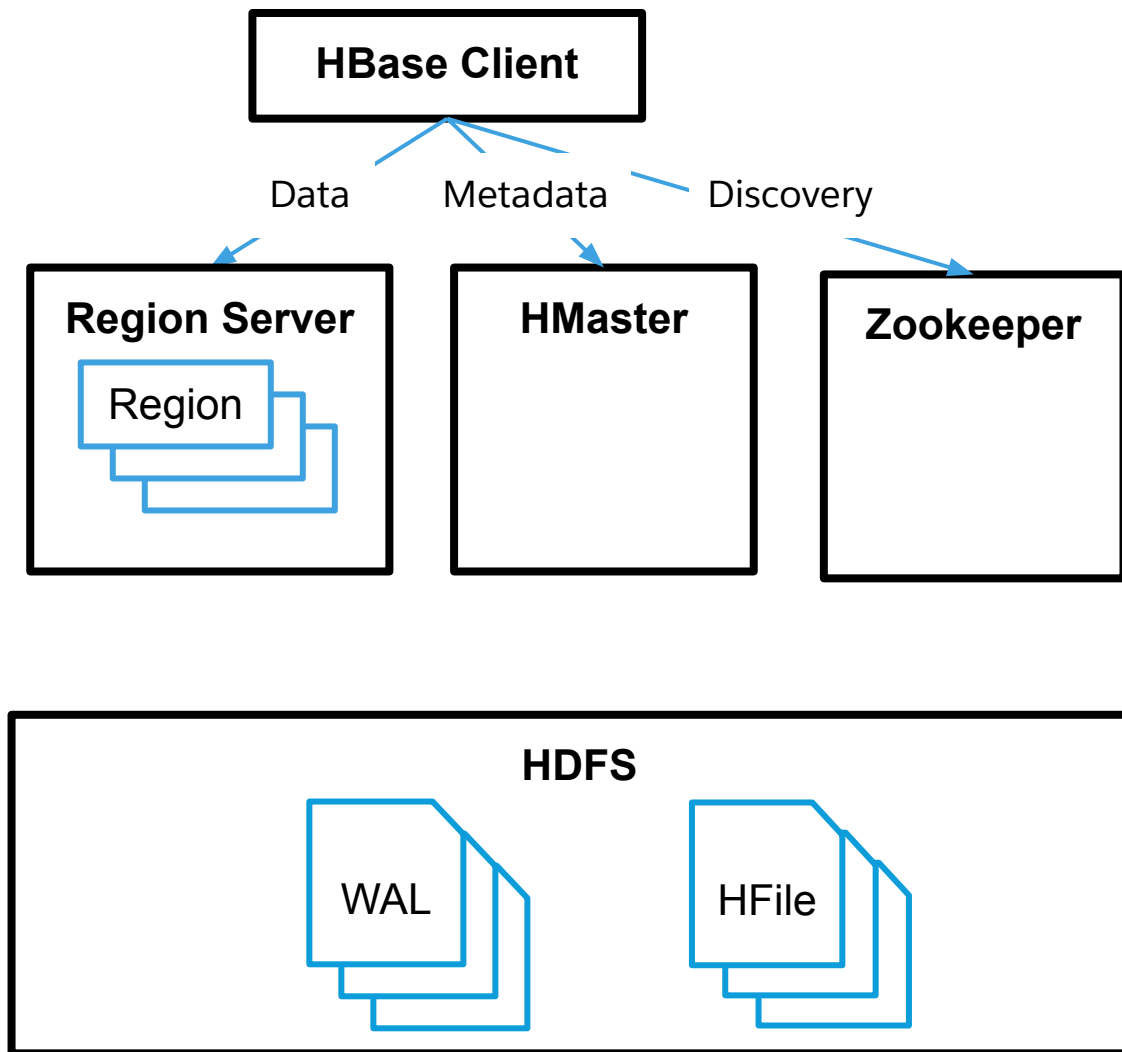
Architecture



HBase Architecture



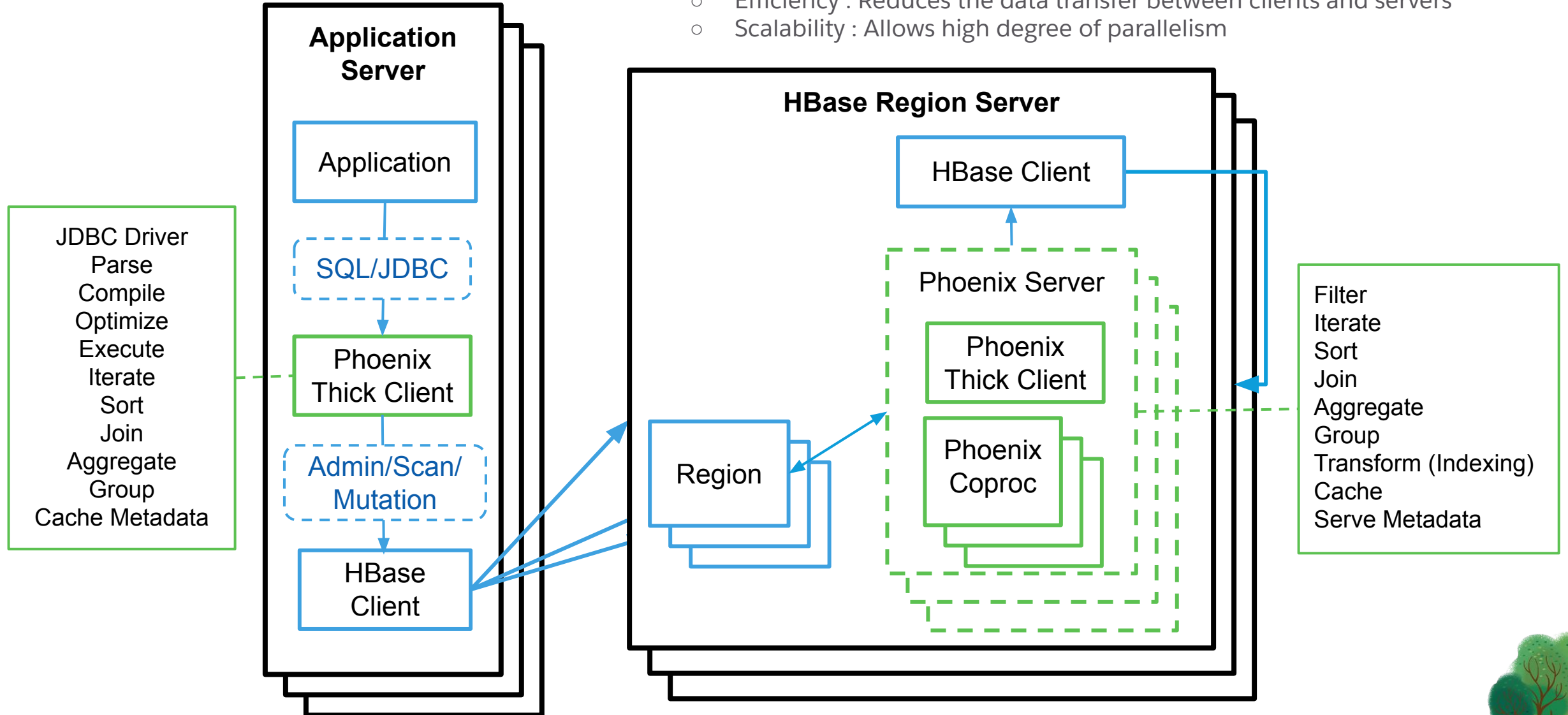
- HBase is a horizontally scalable real-time key-value data store designed for high volume random read/write access
- A table is a sorted map of key-value pairs
- Tables are horizontally divided into regions
- Regions are assigned to nodes called region servers



Phoenix Architecture

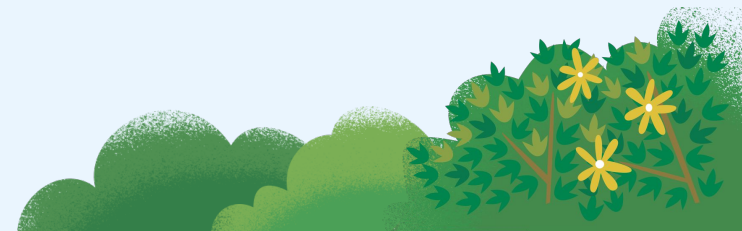
salesforce

- Phoenix is a horizontally scalable relational database and uses HBase as its backend store
- Pushes computation down to Phoenix server (coprocs) for
 - Efficiency : Reduces the data transfer between clients and servers
 - Scalability : Allows high degree of parallelism





Thick Client Problem in Phoenix



Main Problems with Thick Client

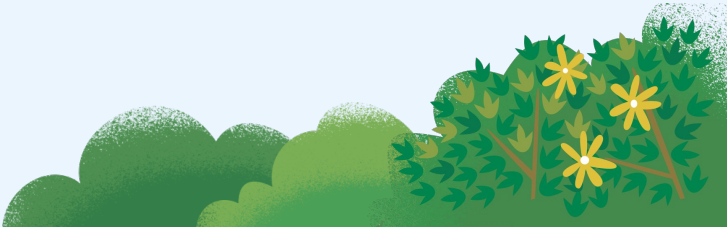


- Restricted release schedule due to tight coupling with core app CI/CD pipeline
 - A Phoenix release has to go through first server and then client upgrade
 - Client upgrade has to be synchronized with the core app release which delays deployment process for 3-6 months
- Performance, scalability and availability issues due to shared compute resources with core app
 - Limited memory, thread pool, and CPU resources makes Phoenix client a bottleneck for
 - Range queries on large tables due to limited parallelism
 - Salting use cases
 - High volume use cases
- Bugs in Phoenix client impact the core app availability





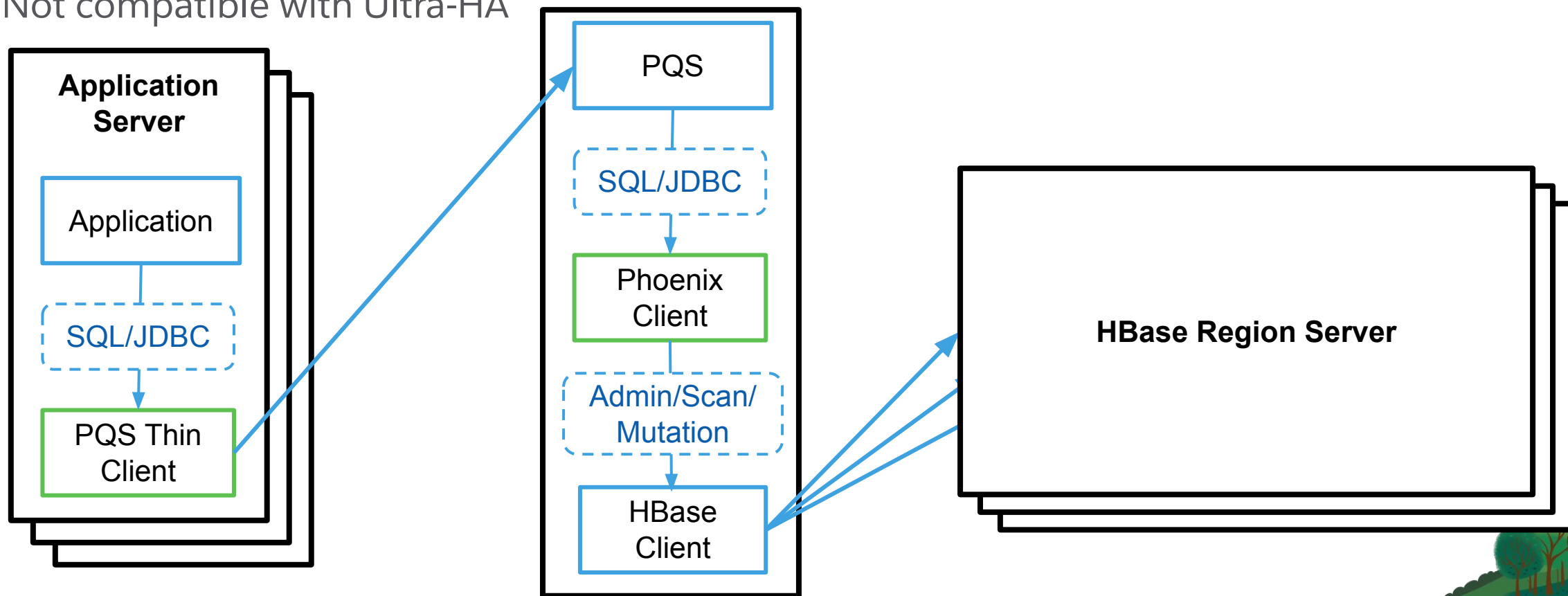
Thin Client



PQS: Tried and Failed



- Phoenix Query Server (PQS) is a known thin client for Phoenix
- A single point of failure
 - Requires building a cluster of PQSes but PQS is not really stateless
- Created connectivity and performance issues
- Not compatible with Ultra-HA



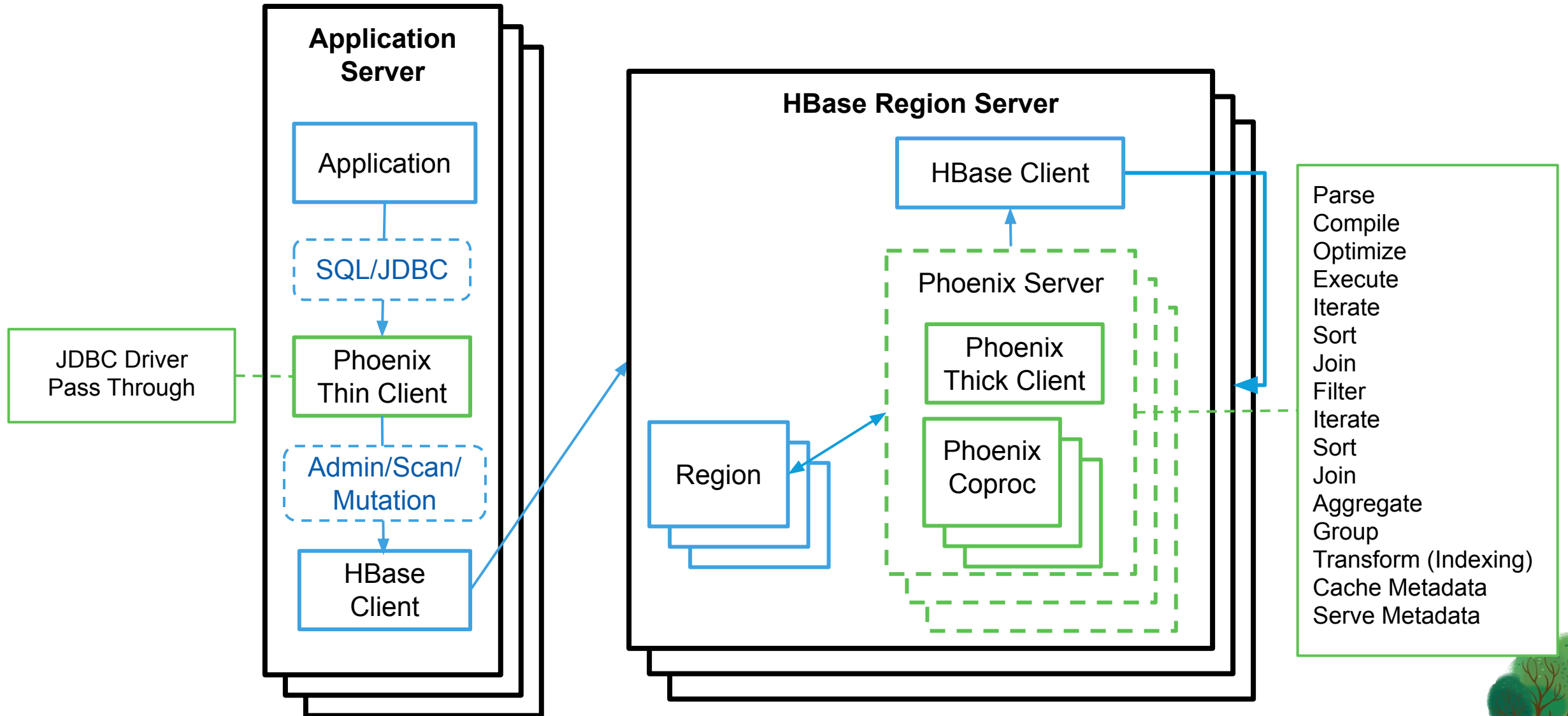
Proposed Solution for Thin Client



- Push not just computation but also entire client functionality to server
 - Currently, we also run the thick client code on region servers
- The Phoenix client runs on the application server becomes a pass through layer
- No new component in the architecture
- No new protocol or public API



Phoenix Architecture with Thin Client



Thin Client Pass Through



- A new table called SYSTEM.CONNECTION
 - The PK of this table is the connection UUID
 - This table is pre split into N number of table regions
 - N can be the maximum number of region servers in a cluster, say 1024
 - With disabled WAL as this table will be always empty on this disk
- Thin client create connections, statements and batches locally
- JDBC execute and execute update calls are translated mutations on the connection table
 - The mutation row key prefix is the connection UUID
 - The statements are passed as mutation attributes
 - The server uses its local thick client to execute statements
 - The mutations are then deleted from the memstore and do not make to the table on disk
- JDBC execute query calls are translated to scans on the connection table
 - The scan start key is the connection uuid
 - The user query is passed as a scan attribute
 - The result of the query is returned with this scan
 - The server uses its local thick client to execute the query



Pros and Cons



Pros

- More freedom from the core app CI/CD pipeline
- No performance, scalability and availability issues due to shared compute resources
- Ultra-HA compatible
- No validate metadata cache RPCs and no metadata transfer for each operation
 - Thick client metadata caching will be strongly consistent on the server side
- Better service protection due to having global state for all client connections
- Better metric collection for client activities

Cons

- Extra compute resources needs to be allocated for the thick client on the region servers
 - This is not due to extra computation but because of shifting compute from app servers to region servers
- Extra hop would slightly increase the latencies
 - This is not specific to this solution as any thin client solution needs an extra hop

